

课程实验报告

信息抽取实验报告

规则系统与 GLiNER 联合抽取技能结构化信息

组号：第 15 组

成员一：张明辉-2023211959

成员二：杨佳怡-2023212446

日期：2026 年 5 月 30 日

摘要

这次实验做的是一个偏工程型的信息抽取系统：从 skills.sh 的技能描述里抽出平台、技术栈、动作类型、目标领域、输出格式和量化指标六类信息。系统没有把所有责任都压给一个模型，而是拆成三层：baseline 负责精确关键字匹配，enhanced-regex 负责 alias 扩展、受控词表归一化和 fallback，enhanced-gliner 再把 GLiNER 接到前面做候选实体提取。

我们将评测集分成 10 条和 24 条两种，分别测量在不同基准下抽取系统的能力，分别比较 baseline、enhanced-regex 和 enhanced-gliner 这三种方法的结果。结果显示，在 10 条种子集上，GLiNER 的 macro F1 仍然略低于 regex (59.00% 对 59.59%)；但扩展到 24 条以后，enhanced-gliner 的 macro F1 提到 75.29%，比 enhanced-regex 的 73.63% 高 1.66 个百分点，micro F1 也从 74.01% 提到 74.65%。

1 引言

这个子系统要解决的问题总体上可以解释为：给一条技能描述，自动整理出它服务哪个平台、用什么技术、做什么动作、面向什么场景、最后产出什么格式。检索系统拿到这样的结构化结果后，界面展示和后续统计都会轻松很多，后面如果接问答或推荐，也不用每次都从长文本里硬抠关键词。

数据跟信息检索程序一样还是同一批 1000 条 skills.sh 记录。字段除了 skill_name、description、category 这些主键，还包含外置的 SKILL.md 文本路径。当前快照里 917 条能读到外置技能文档，83 条缺 SKILL.md，203 条缺简介。文本语言整体偏英文，标题和描述同时含 CJK 的记录非常少，所以抽取系统里专门加了一个 english_only_bias 开关，避免 GLiNER 在非英文文本上乱猜。

这次补实验最关心四件事：第一，baseline 和 enhanced-regex 到底各修掉了哪些问题；第二，把 10 条种子标注扩成一份更像样的评测集以后，GLiNER 的收益会不会改观；第三，如果模型命中了 span 但最后还是没被系统接住，瓶颈究竟出在词表、阈值还是后处理；第四，实验环境会不会直接改写结论。

2 数据获取与预处理

2.1 抓取与 checkpoint 流程

IE 系统用的数据不是另外造的，而是直接吃 paqu.py 写下来的共享数据集。抓取流程和 IR 报告一致，这里仍然给出一张图，方便后面解释为什么有的文本来自 description，有的来自外置 SKILL.md。

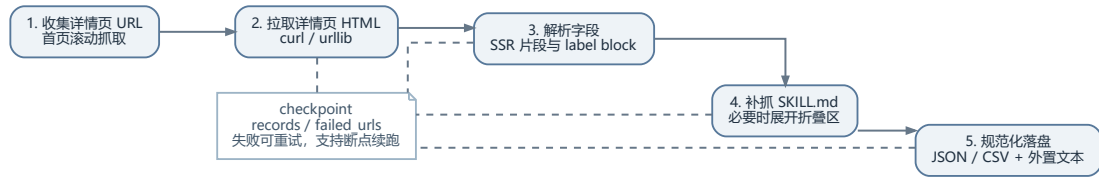


图 1: IE 使用的数据准备链路

2.2 字段与文本来源

抽取阶段真正关心的是表 1 这些字段。它们分别用于构造输入文本、辅助规则匹配和记录结果来源。

表 1: IE 子系统直接使用的主要字段

字段名	作用	非空条数
skill_name	事件标题, 构造 full_text 与 gliner_text	1000
description	短摘要, GLiNER 的主要上下文之一	797
category	类别先验, 常和领域词一起进入输入文本	1000
skill_path	长文本主来源, 覆盖率最高	917
skill_raw_path	保留原始换行, 便于正则抽取 metrics	917
detail_url	回看样本和误差分析时的定位键	1000

代码里读文本的优先级是

```
external_skill_text > skill_md_raw_text > skill_md > description.
```

这一步很关键。像 maishou 这种技能, 如果只看简介, 平台名单根本不完整; 而像 domain-authority-auditor 这种长规则说明文档, 如果直接把整段都丢给 baseline, 又会触发一堆过宽的动词和领域词。

2.3 预处理: 文本包与语言偏置

抽取前, 系统会为每个文档构造三个视图:

- full_text: skill_name + category + extraction_text, 供完整规则匹配使用;
- gliner_text: 压缩过的聚焦文本, 避免超长输入把 GLiNER 拖慢;
- fallback_text: GLiNER 失败或跳过后回退使用的文本。

语言偏置判断写得很直白。设文本里的 ASCII 字母数为 n_{ascii} ，CJK 字符数为 n_{cjk} ，则

$$\text{english_dominant}(x) = \begin{cases} 1, & n_{\text{cjk}} = 0, \\ 1, & n_{\text{ascii}} \geq 1.5 n_{\text{cjk}}, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

当 `english_only_bias=true` 且式 (1) 判成 0 时，GLiNER 直接跳过，系统退回规则路径。

这样做的原因有两个：一个是模型本身的倾向。GLiNER 的预训练语料以英文为主，中文 skill 描述丢进去后，模型经常把中文短语拆成无意义的单字片段当实体吐出来，或者给完全无关的候选标上中等置信度。这些输出既不是规则层能接住的规范值，也不是人工标注时会认的关键信息，不加筛选会让 `gliner_unknown` 里多出大量中文噪声，干扰后续误差分析。与其让模型在它不擅长的文本上硬猜，不如直接退回规则——至少规则行为是透明的，错也知道错在哪。

另一个是当前数据的实际分布。1000 条 skill 记录里，文档主体几乎全是英文。对少量的中文 skill 来说，GLiNER 跳过的代价极小——它们本来占全部数据的比例也不高，退化成纯正则对整体指标的影响微乎其微。反过来，如果把 `english_only_bias` 关掉，用 GLiNER 硬跑全部 1000 条，中文噪声反而会把评测结果搅乱，掩盖 GLiNER 在英文文本上的真实收益。

说白了，这个开关的作用不是“挑语言”，而是“在模型能力和数据分布的限制下，把 GLiNER 的算力花在它最可能做对的文本上”。它是一个工程决策，不是语言偏好。

3 核心算法

3.1 三阶段结构

这套 IE 系统不是单一模型，而是三层递进关系。图 2 画出了它们的区别。

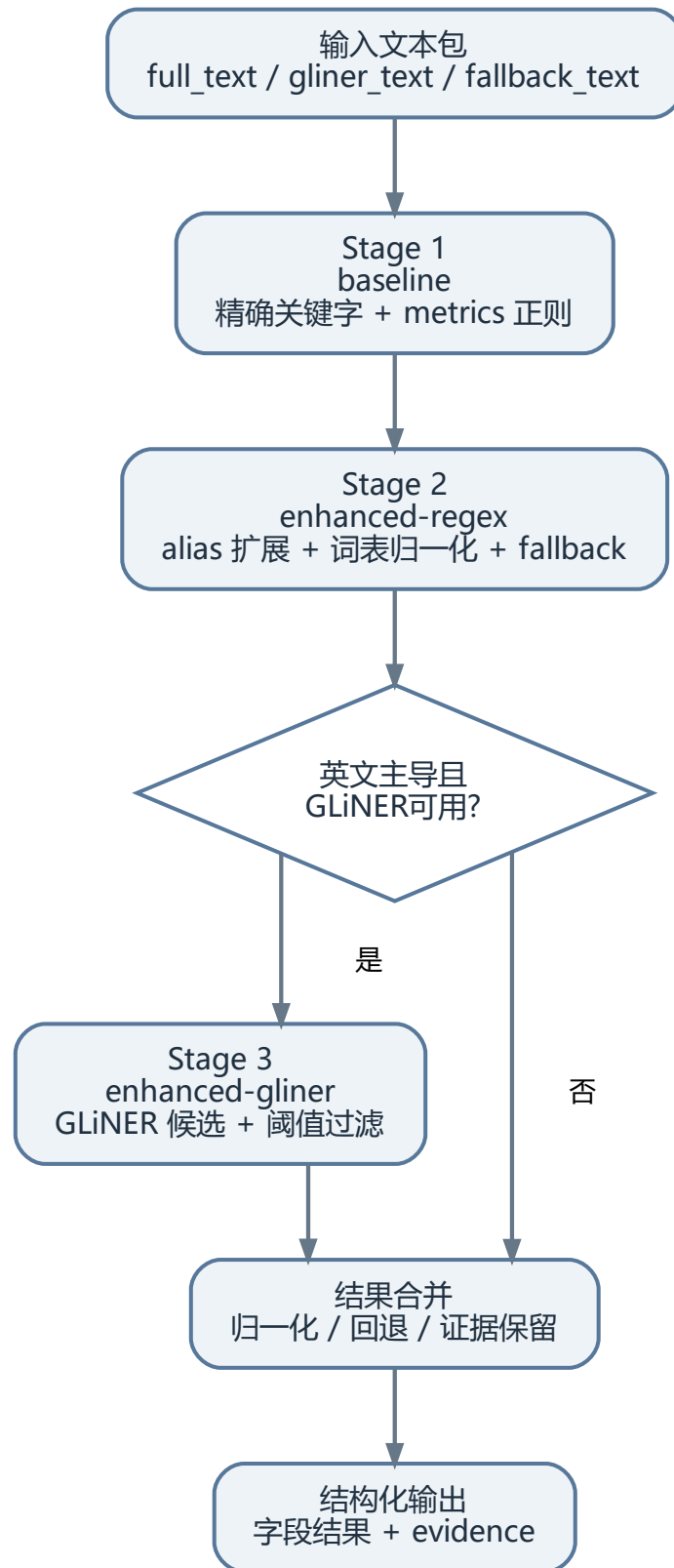


图 2: IE 三阶段结构

3.2 baseline: 精确关键字与 metrics 正则

baseline 对五个枚举字段都走 `\bkeyword\b` 形式的正则精确匹配，配置项来自 `ie_config.json`。这样做的好处是行为很可审计，坏处也同样明显：词表一旦偏窄，FN 会很多；词表一旦偏宽，FP 也会很多。

`metrics` 字段单独处理，不靠关键字表，而是靠四组模式去抓“数字 + 单位”：

表 2: metrics 字段使用的主要正则模式与示例

模式	典型命中示例
<code>(\d+)[-\\s]*(signal criteria item ...)</code>	5-signal, 12-criteria
<code>(?:across over with covering spanning)\\s+(\d+)\\s+...</code>	across 8 dimensions
<code>(\d+)\\s*[---]\\s*(\d+)\\s*(?:scale score range rating)</code>	1-10 scale
supports five modules 这类英文 数字模式	supports five modules

3.3 enhanced-regex: alias 扩展与受控词表归一化

第二层增强主要做两件事。第一件是 action alias 扩展，例如 `analysis`、`analytics`、`analyzer` 都归到 `analyze`，`writer`、`writing`、`drafting` 都归到 `create`。第二件是受控词表归一化，顺序是

$$\text{exact} \rightarrow \text{folded} \rightarrow \text{alias} \rightarrow \text{fuzzy} \rightarrow \text{unknown}. \quad (2)$$

这层增强是这次实验里真正的主力。拿 `maishou` 来说，baseline 只能抓到 1688，enhanced-regex 则能把 `taobao`、`tmall`、`jd.com`、`pinduoduo`、`vip.com` 一次性补齐。

3.4 GLiNER 集成与阈值门控

第三层把 GLiNER 接进来做候选实体提取。它在系统里的位置不是“直接产出最终答案”，而是“先提 span，再让规则层决定要不要收下”。整条链路见图 3。

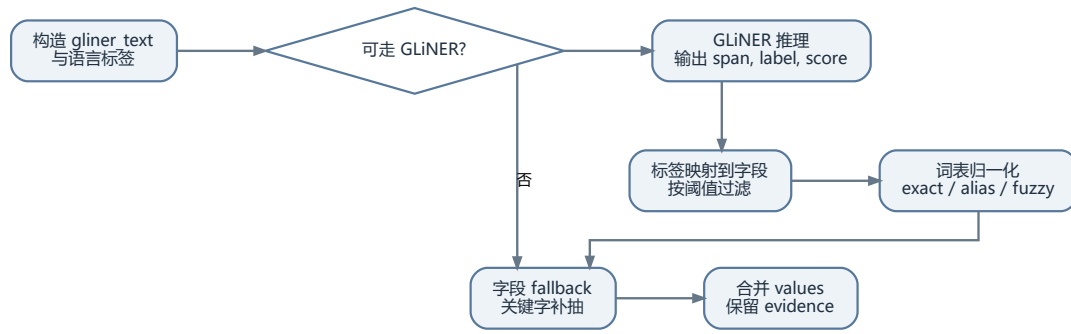


图 3: IE 使用的数据准备链路

字段阈值是手工设的，表 3 给出当前配置。

表 3: GLiNER 各字段的置信度阈值

字段	阈值
platforms	0.4
languages	0.3
action_types	0.5
target_domains	0.5
output_formats	0.3
metrics	0.6

这几个数不是同一套逻辑定下来的，每个字段的阈值背后有自己的理由。

`languages` 和 `output_formats` 设得最低（0.3）。技术名词和格式名在文本里通常以专有名词形式出现（Python、JSON、CSV），GLiNER 一旦认出，置信度一般都不低；但词表偏小时，把阈值压到 0.3 是为了尽量不漏——格式名就 22 个词，再卡严一点多个样本的输出直接为空。

`platforms`（0.4）取中间值。平台名同样是专有名词，但 GLiNER 在长文档里偶尔会把一些不相关的服务名（如某段篇幅中提到的 AWS、Heroku）也标出来，置信度在 0.3–0.4 区间。设 0.4 能滤掉大部分弱信号，同时保留真正作为核心平台出现的实体。

`action_types` 和 `target_domains` 设到 0.5，是六个字段里最保守的两档。动作动词和领域词在技能描述里太密集了——一篇 SEO 工具文档里，`analyze`、`generate`、`report`、`audit` 可能各出现十几次，GLiNER 很容易对这些词标出 0.4 甚至 0.45 的置信度，但它们只是在描述上下文，不是技能本身的核心动作。阈值不抬高，FP 会从 `action_types` 和 `target_domains` 两个字段同时灌进来。

`metrics`（0.6）最严。原因很简单：数字天然难判——GLiNER 会把 `5 modules`、`12 signals`、`over 10` 这些东西全当成 metric 标出来，置信度还不低。但实验里真正该抽的指

标是”多少星””多少安装量”这类有明确单位的数值。把阈值拉到 0.6，是为了让 GLiNER 只提供它非常有把握的候选，剩下的靠正则去兜——正则抽 metrics 的模式是精确写死的，precision 比 GLiNER 靠谱得多。

形式上可以写成

$$\text{accept}(x, f) = \begin{cases} 1, & \text{score}(x, f) \geq \tau_f, \\ 0, & \text{score}(x, f) < \tau_f. \end{cases} \quad (3)$$

如果一个 span 过了阈值，但归一化后找不到合法词表项，系统不会静默吞掉，而是把它记成 `gliner_unknown`。这一点对误差分析很有用。

3.5 增强版抽取流程伪代码

算法 1: enhanced 抽取主流程

输入: 文档 d

输出: 抽取结果 E 与证据映射 V

```

1  $B \leftarrow \text{BUILDTEXTBUNDLE}(d)$ 
2 if variant = baseline then
3   return EXACTKEYWORDEXTRACT( $B.\text{full\_text}$ )
4 if  $B.\text{gliner\_eligible}$  且模型可用 then
5    $P \leftarrow \text{PREDICTGLINER}(B.\text{gliner\_text})$ 
6    $H \leftarrow \text{PROJECTBYFIELD}(P, \tau_f)$ 
7 else
8    $H \leftarrow \emptyset$ 
9 foreach 枚举字段  $f$  do
10  if  $H[f]$  有可归一化命中 then
11     $E[f] \leftarrow \text{NORMALIZE_TO_VOCAB}(H[f])$ 
12     $E[f] \leftarrow \text{MERGEFALLBACK}(E[f], B.\text{fallback\_text})$ 
13  else
14     $E[f] \leftarrow \text{FALLBACKEXTRACT}(f, B.\text{fallback\_text})$ 
15  $E[\text{metrics}] \leftarrow \text{METRICREGEX}(B.\text{full\_text}, H[\text{metrics}])$ 
16 return ( $E, V$ )

```

4 实验设计与过程

4.1 实验环境

操作系统	Windows 10
Python	3.11.5
包管理	conda
关键依赖	scikit-learn (TF-IDF)、numpy、Flask (Web 界面)
GLiNER 模型	urchade/gliner_multi-v2.1 (mdeberta-v3-base 骨干)
硬件	CPU-only, 无 GPU

4.2 标注集与评测办法

实验将评测拆成了两层。分别为 `ground_truth.json`，它只有 10 条 skill，可以当作一个种子集，和 `ground_truth_expanded.json`，扩到 24 条，并且保证这 24 条都能在当前 `skills\data.json` 里对齐到文档。扩展集里五个自动评测字段的支持度分别是：`platforms=8、languages=9、action_types=22、target_domains=16、output_formats=11`。

标注时定了一条规则：只记当前 schema 里确实存在、而且描述或 `SKILL.md` 里明确出现的核心值。实现细节、外围术语和过度泛化的词不纳入真值。这样做的目的不是把标签堆得越多越好，而是让 Precision / Recall 反映“系统有没有抓到关键结构化信息”，避免把文档里顺手提到的背景词也当成必须命中的目标，从而拉偏评测结果。由于 `metrics` 的值是 `{value, unit}` 字典列表，直接做集合交并不合适，自动评测仍然将它排除在外。

Precision、Recall、F1 的定义仍然是集合式写法：

$$P = \frac{|Y \cap \hat{Y}|}{|\hat{Y}|}, \quad (4)$$

$$R = \frac{|Y \cap \hat{Y}|}{|Y|}, \quad (5)$$

$$F1 = \frac{2PR}{P + R}, \quad (6)$$

其中 Y 是人工标注集合， $\hat{Y}$ 是系统抽取集合。

除了原来的 macro 指标，我还补了一组 micro 指标，把所有 skill-field-value 当成一个整体去累计 $TP/FP/FN$ 。这样做的原因很简单：24 条样本虽然比 10 条好很多，但字段支持度仍然不均衡。只看 macro F1 时，某些小字段上的一次命中或一次误判会被放得很大；加上 micro F1 以后，就能看出整体值集合上的真实得失。

另外，我又加了一层“regex $\rightarrow$ gliner 改变了哪些 skill-field”的对比统计。这个统计不直接看总分，而是看 GLiNER 到底改没改最终结果、改动落在哪个字段、是修好还是修坏。这个视角对判断 GLiNER 的实际贡献比一句总 F1 更有用。

4.3 运行命令

实验中使用五条命令完成抽取、评测、变体对比、报告生成和诊断分析。下面逐一说明每条命令的输入、输出和在实验链路中的位置。

4.3.1 extract

```
python ie_system/skills_ie_system.py extract --variant enhanced
```

对全量 1000 条 skill 记录执行抽取。流程为：加载数据集 → 按 `--variant` 选择抽取路径（baseline 走纯正则，enhanced 走 GLiNER + fallback）→ 逐文档构造文本包 → 输出每个文档的字段值、证据链和调试载荷。`--variant` 默认就是 enhanced，不加这个参数行为一致；`--limit N` 可以只跑前 N 条，调试时常用。

结果写入三处：`ie_system/output/extraction_results.json`（完整抽取结果，每条包含六个字段的值和 evidence）、`ie_system/runtime/extraction_report.json`（覆盖率与证据来源统计）和 `ie_system/runtime/project_state.json`（运行状态记录）。这条命令是所有后续操作的前置——`evaluate`、`compare`、`report` 都依赖它的输出。

4.3.2 evaluate

```
python ie_system/skills_ie_system.py evaluate --variant enhanced --eval-set  
ground_truth.json
```

在指定评测集上对单一变体的抽取结果做自动评估。流程为：先跑 `extract_all()` 得到当前变体的全量结果，再加载 `--eval-set` 指定的 ground truth 文件，逐 skill 比对系统输出和人工标注，按字段计算 Precision、Recall、F1（macro 平均）。输出写入 `ie_system/runtime/evaluation_report.json` 和 `ie_system/runtime/eval_report.json`。

和 `compare` 的区别：`evaluate` 只看一个变体在指定评测集上的绝对分数，不涉及变体间的横向对比。适合在调参后快速验证一个改动（比如调了某个字段的阈值、或者改了一组别名规则）有没有让分数变好。

4.3.3 compare

```
python ie_system/skills_ie_system.py compare --eval-set ground_truth.json
```

同时跑 baseline 和 enhanced 两个变体，在同一个评测集上横向对比。内部流程是：分别用两个变体各跑一遍全量抽取 → 分别和 ground truth 比对 → 输出分字段的 P/R/F1 差异表，以及每个 skill 在两个变体下的结果差异。报告还附带覆盖率对比（每个变体各抽出了多少字段、多少文档一条信息点都没抽出来）和证据来源分布对比。

这条命令回答的是“加了增强层到底改了些什么”——哪些字段的 F1 涨了、哪些跌了、哪些 skill 在两个变体下输出一模一样。输出写入 `ie_system/runtime/comparison_report.json` 和同名的 Markdown 文件。

4.3.4 report

```
python ie_system/skills_ie_system.py report --variant enhanced
```

生成全量抽取的统计报告。和 `extract` 自带的那份 `report` 是同一套逻辑，但可以独立运行，不重新做抽取——直接读已有的 `extraction_results.json` 做汇总。报告内容包括：每条文档的信息点数量分布、六个字段各自的覆盖率、全局证据来源分布（`gliner_direct`、`gliner_unknown`、`keyword_fallback` 等各出现多少次）、以及每个字段按证据来源拆开的 TOP-5 来源统计和抽样示例。

这条命令在误差分析阶段用得最多：看 `gliner_unknown` 的绝对数量和来源分布，判断 GLiNER 的瓶颈是在词表覆盖不够还是在字段归类不对。

4.3.5 diagnose-eval

```
conda run -n yolov82 python ie_system/skills_ie_system.py diagnose-eval \
  --eval-set ground_truth_expanded.json \
  --compare-eval-set ground_truth.json \
  --skip-full-compare
```

这是本次实验专门加的一条诊断命令，用来把 legacy 10 条和 expanded 24 条两套评测集的结果放在一起看。它做的事情比 `compare` 更综合：在两套评测集上分别跑 `baseline`、`enhanced-regex`、`enhanced-gliner` 三个变体 → 汇总 `macro` 和 `micro` 两组 P/R/F1 → 逐 skill-field 追踪 `regex` → `gliner` 的改动（增益/退化/不变）→ 统计 `gliner_unknown` 的全量分布并按字段和原因分类。

`--skip-full-compare` 跳过 1000 条全量 `enhanced` 的 `regex` vs `gliner` 逐条对比（省时间），只保留评测集上的交叉比较和 `gliner_unknown` 分类。输出写入 `ie_system/runtime/eval_diagnostic_report.json` 和 `ie_system/runtime/eval_diagnostic_summary.md`，后者可以直接当误差分析的起点读。

这条命令必须用 `conda run -n yolov82` 是因为 GLiNER 的 `torch` 依赖只在这个环境里正常——`base` 环境会报 `c10.dll` 初始化失败，导致 GLiNER 路径退化。命令名前面挂 `conda run` 不是规范做法，但在当前环境下是最可靠的保证。

5 实验结果与对比分析

5.1 三阶段总表

表 4 把两套评测集一起列出来。为了不被单一口径带偏，我同时保留了 `macro` 和 `micro` 两组指标。`macro` 更强调样本级平均，`micro` 更强调所有字段值累积后的整体得失。

表 4: IE 在 legacy 10 条与 expanded 24 条评测集上的整体结果

评测集	变体	Macro P	Macro R	Macro F1	Micro F1
legacy 10	baseline	0.3321	0.4348	0.3766	0.3478
legacy 10	enhanced-regex	0.5823	0.6102	0.5959	0.6013
legacy 10	enhanced-gliner	0.5712	0.6102	0.5900	0.6065
expanded 24	baseline	0.3329	0.5024	0.4005	0.3941
expanded 24	enhanced-regex	0.6937	0.7845	0.7363	0.7401
expanded 24	enhanced-gliner	0.7046	0.8083	0.7529	0.7465

这张表正好说明，小评测集确实会影响结论。10 条种子集上，GLiNER 的 macro F1 比 regex 低 0.59 个百分点；但扩到 24 条以后，enhanced-gliner 的 macro F1 到了 75.29%，比 enhanced-regex 高 1.66 个百分点，micro F1 也高 0.64 个百分点。也就是说，原来那句“GLiNER 没提升”并不稳，它更像是小样本条件下的局部现象。

不过另一半结论也要保留：baseline $\rightarrow$ regex 的跃升才是主干，regex $\rightarrow$ gliner 的提升比较窄，不是所有字段都一起涨。

5.2 分字段 F1 对比

图 4 用的是 expanded 24 条评测集上的分字段 F1。这里能看出 GLiNER 到底补了什么：平台和语言字段在 regex 阶段已经很高，GLiNER 基本不再改动；动作和目标领域也几乎不动；真正被它拉起来的是 output_formats，从 0.6154 提到 0.6969。

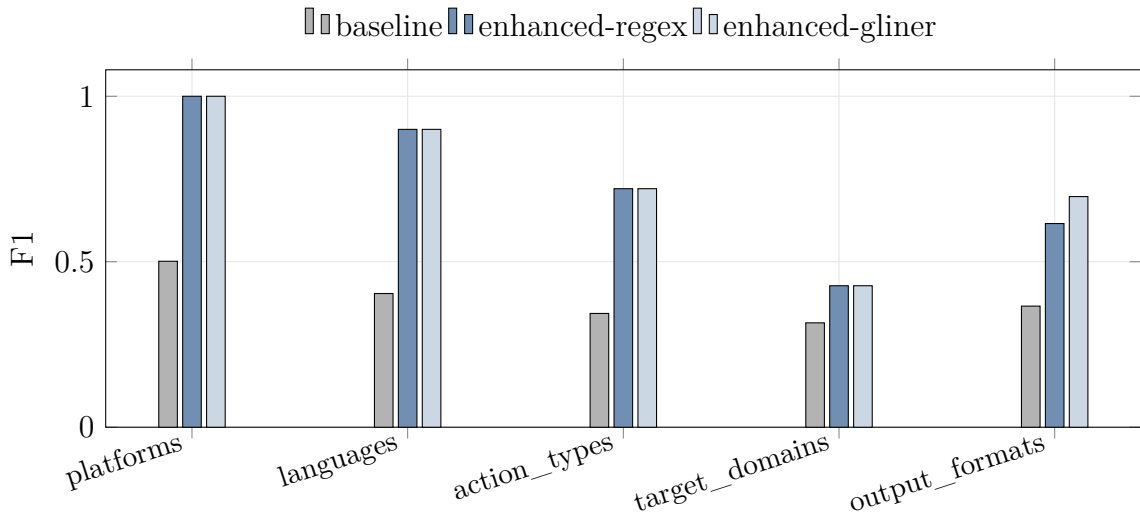


图 4: expanded 24 条评测集上的分字段 F1: regex 阶段已经把大部分字段抬到较高水平，GLiNER 的新增收益集中在 output_formats，其余字段基本保持不变。

从样本级变化去看，expanded 24 条里 regex $\rightarrow$ gliner 一共改动了 4 个 skill-field，对应 3 次增益、1 次退化。3 个增益分别是：

- `manim-composer`: 补出了 `output_formats=markdown`, 单字段 F1 从 0 提到 1;
- `frontend-design`: 把漏掉的 `css` 补回来, 单字段 F1 从 0.8 提到 1;
- `docx`: 补出 `jpg`, 单字段 F1 从 0.6667 提到 0.75。

唯一一次退化出现在 `pptx`, GLiNER 多给了一个 `jpg`, 把单字段 F1 从 0.6667 拉到 0.5。说明 GLiNER 现在更像“能修一小部分漏项, 但偶尔也会顺手带入边界模糊值”的状态。

5.3 全量 1000 条覆盖率

为了不只盯着 24 条标注样本, 我又看了全量 `extraction_report.json`。当前 enhanced 输出在 1000 条文档上的字段覆盖率是:

- `platforms`: 27.8%
- `languages`: 45.6%
- `action_types`: 90.7%
- `target_domains`: 92.7%
- `output_formats`: 47.9%
- `metrics`: 13.7%

证据来源分布也挺有意思。全局最多的是 `keyword_fallback`(8390 条), 其次是 `gliner_unknown`(4108 条)、`alias_fallback`(2168 条)、`gliner_direct`(1765 条)、`gliner_normalized`(655 条) 和 `gliner_reassigned`(429 条)。说明 GLiNER 不但在跑, 而且一部分候选已经能通过重定向规则落到正确字段里。

5.4 误差分析

这部分我只挑最典型的坑, 不把所有 bad case 都平铺出来。

1. 关键字过宽带来的 FP `domain-authority-auditor` 在 baseline 下的 `action_types` 特别典型。标注期望只有 `audit/detect/evaluate`, 系统却能抽出 `build`、`compare`、`generate`、`report`、`review`、`validate` 一大串。问题不在正则写错, 而在这类技能文档本来就像操作手册, 动词太密了。只要看到词就收, Precision 一定扛不住。

2. 词表覆盖不够, FN 还是会留下 `write-xiaohongshu` 的 `target_domains` 期望是 `content/copywriting/media`, 结果当前输出还是偏到 `ai/frontend`; `domain-authority-auditor` 和 `entity-optimizer` 也会把 SEO 语境里的内容词吸到 `content` 上, 却不一定稳稳落到 `seo` 本身。这说明词表还不够细, 尤其是“领域词”和“背景语境词”之间的边界还比较糊。

3. GLiNER 命中了，但归一化层没有接住 对最近一轮成功的全量 enhanced 输出做分类后，`gliner_unknown` 一共有 4108 条，其中 `platforms` 1819 条、`languages` 1393 条、`output_formats` 468 条、`action_types` 412 条、`target_domains` 16 条。相比修复前的 5062 条，绝对值减少了 954 条。按原因拆开看，除了大量难以自动归类的长尾项之外，最值得关注的几类是：

- 剩余词表缺项：`ccc`（15 次）、`git`（12 次）、`BigQuery`（9 次）、`OG images`（3 次）、`music`（3 次）、`images`（3 次）这类实体仍然没有稳定入口；
- 字段串位：这一类从 231 次降到 104 次，说明重定向规则已经起效，但 `Exa/BigQuery/HTTP` 这类 `span` 还会在 `languages` 或 `output_formats` 之间打架；
- schema 边界问题：`iOS/Android/macOS/Linux/Windows` 更像运行平台或操作系统，但当前 schema 没有这类字段，只能堆到 `platforms` 的 `unknown` 里；
- 动作名词化：`authentication`、`error handling`、`navigation`、`rate limiting` 这类短语被打到 `action_types`，但既不是现成 `canonical verb`，也不该原样收下。

这组统计和样例一起看，GLiNER 当前最大的障碍已经不是“能不能看见实体”，而是“看见以后，schema 有没有地方让它落下来”。

4. 真实踩坑：环境错了会把结论直接带偏 这轮排查里最容易忽略、但影响最大的坑其实不是规则，而是环境。如果直接用 `base` 环境运行，调试载荷里的 `mode` 会变成 `gliner_unavailable`，`model_error` 指向 `torch c10.dll` 初始化失败，整条 `enhanced-gliner` 路径会退回纯规则 `fallback`。换句话说，实验环境一旦没对齐，最后看到的“`gliner` 没提升”可能根本不是模型结论，而是模型没跑起来。

5. GLiNER 也会带来新的误判 `pptx` 就是一个例子。`regex` 阶段已经抓到 `pptx+xml`，GLiNER 再加了一个 `jpg`，结果单字段 F1 反而从 0.6667 掉到 0.5。说明 GLiNER 现在更像“有时能补回漏项，也有时会顺手带入边界模糊值”的状态。

6 总结与展望

这轮补实验之后，我对 IE 子系统的判断比之前更具体了。第一，原来那句“GLiNER 没提升”确实说早了。把评测集从 10 条扩到 24 条、再把实验环境固定到 `yolov82` 以后，GLiNER 的收益可以被测出来，而且主要落在 `output_formats` 这一类 `regex` 容易漏边角的字段上。第二，`regex` 增强层依然是主力。真正把系统从“见词就收”拉到可用区间的，是 `alias`、归一化和 `fallback`，不是模型单独救场。

局限也同样清楚。扩展后的 24 条评测集已经比 10 条靠谱很多，但字段支持度还是不平衡；GLiNER 的增益集中在少数字段，还不算全面；即便修过一轮，全量输出里仍然有 4108 条 `gliner_unknown`，说明 schema 和词表比模型本身更像当前瓶颈。后面如果继续做，

我会优先补四件事：第一，继续扩平台、语言和输出格式词表，把 `ccc`、`git`、`BigQuery`、`OG images` 这类剩余高频缺项收进去；第二，把 ‘LLM 服务名’、‘运行平台’、‘文档格式描述词’ 分成更清晰的 schema，而不是全塞进现有字段；第三，继续收紧字段重定向规则，避免 `pptx` 这种“补回一项又多带一项”的副作用；第四，把 `metrics` 单独做一套更细的评测，不再只停在“能不能抽出来”。